AIMLZG523
MLOps

Assignment 1
MLOps Experimental Learning: End-to-End ML Model Development, CI/CD, and Production Deployment Experimental Learning

Devang Kamble
2025AE05173

Before we start with the detailed report, these are all the quick-links of our assignment:

- Code Demo Video - Drive Link
- Code Repository - GitHub Link
- Model Notebook Link - GitHub Link
- Deployments
  - Application UI - ui-mlops-assign1.devangk.dev
  - Application API - api-mlops-assign1.devangk.dev
  - Grafana Dashboard - dash-mlops-assign1.devangk.dev
  - Prometheus Query - metrics-mlops-assign1.devangk.dev
- Architecture Diagrams
  - Model Architecture - Image
  - System Architecture - Image
  - Pipeline Architecture - Image

1. Summary

This project builds a complete MLOps workflow around a simple prediction task: given 13 clinical measurements of a patient, estimate whether heart disease is present. The data is the Cleveland subset of the UCI Heart Disease dataset. The model matters less than the workflow around it, so the focus is on making every step repeatable and automated: training and experiment tracking, automated testing, packaging in containers, deployment to a server, and monitoring of the running service.

The pipeline uses scikit-learn for modelling, MLflow for experiment tracking, GitHub Actions for CI/CD, a Flask API and a Streamlit UI packaged with Docker, and Prometheus, Loki and Grafana for monitoring. The selected model is a logistic regression with a nested cross-validation ROC-AUC of 0.891 and a held-out test ROC-AUC of 0.951. The quick links at the top of this report give the code, the demo video and the live deployments.

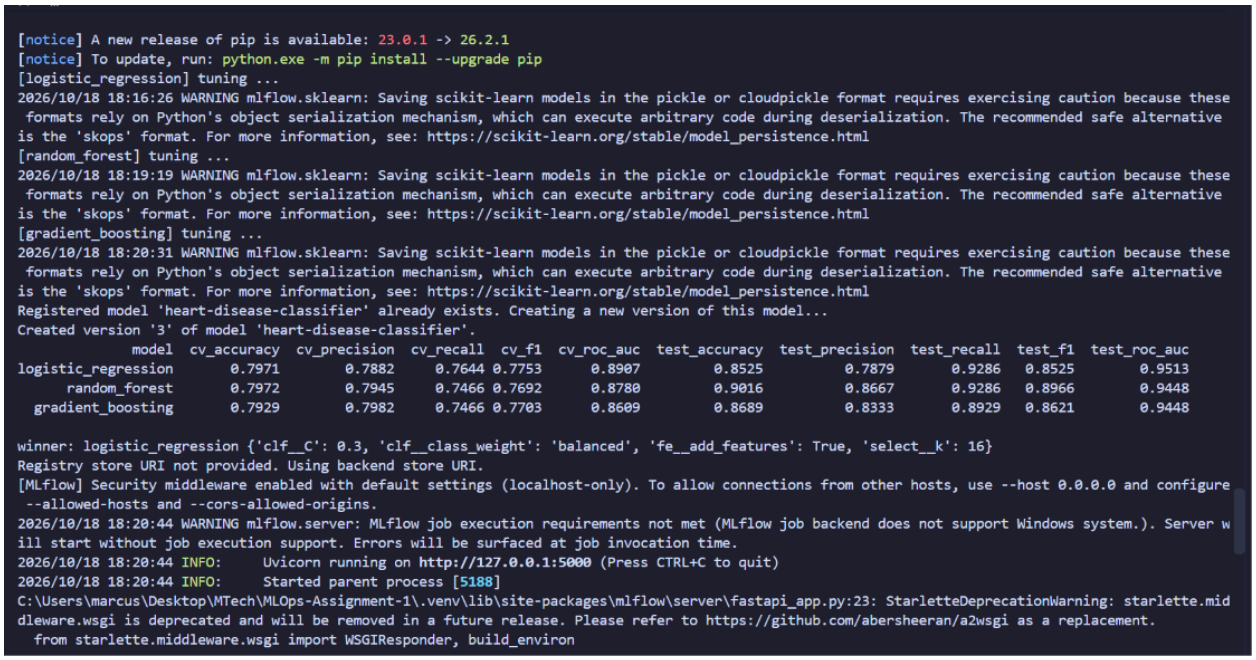
Since I already had my VPS server of my own, I chose this approach of pipeline - deploy everything onto it and make the entire pipeline CI/CD enabled. We first developed and ran the experiment on Prayogshala and then extended to the deployment.

2. Setup

The project needs Python 3.10, Docker with the Compose plugin and Git. Everything below is run from the repository root, and each step depends only on the one before it.

- First setup a Python 3.10 virtual environment
- Install the dependencies with pip install -r requirements.txt. The versions are pinned, so the environment matches the one used in CI.
- Train the model with python -m src.train. This downloads the data, tunes and evaluates the three candidate models, logs every run to MLflow and saves the winner to models/heart_model.joblib. The results can be browsed with MLflow ui --backend-store-uri sqlite:///mlflow.db.
- Run the checks with pytest and flake8 src tests (you would have to run pip install -r requirements-dev.txt), the same commands CI runs.

- Start the whole stack with docker compose up -d --build. This brings up the API (port 7201), the UI (7202), Prometheus (7203), Loki (7204) and Grafana (7205).



Running the scripts

3. Data and EDA

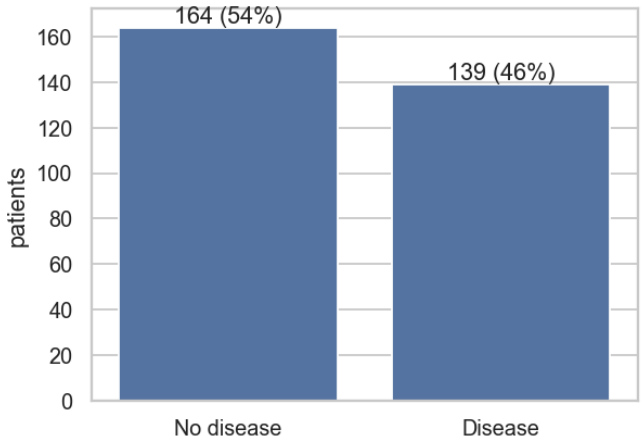
The dataset has 303 patient records and 13 features: ‘age’, ‘sex’, chest pain type (‘cp’), resting blood pressure (‘trestbps’), cholesterol (‘chol’), fasting blood

sugar (‘fbs’), resting ECG result (‘restecg’), maximum heart rate (‘thalach’), exercise-induced angina (‘exang’), ST depression (‘oldpeak’), the slope of the ST segment (‘slope’), the number of major vessels coloured by fluoroscopy (‘ca’), and a thalassaemia test result (‘thal’).

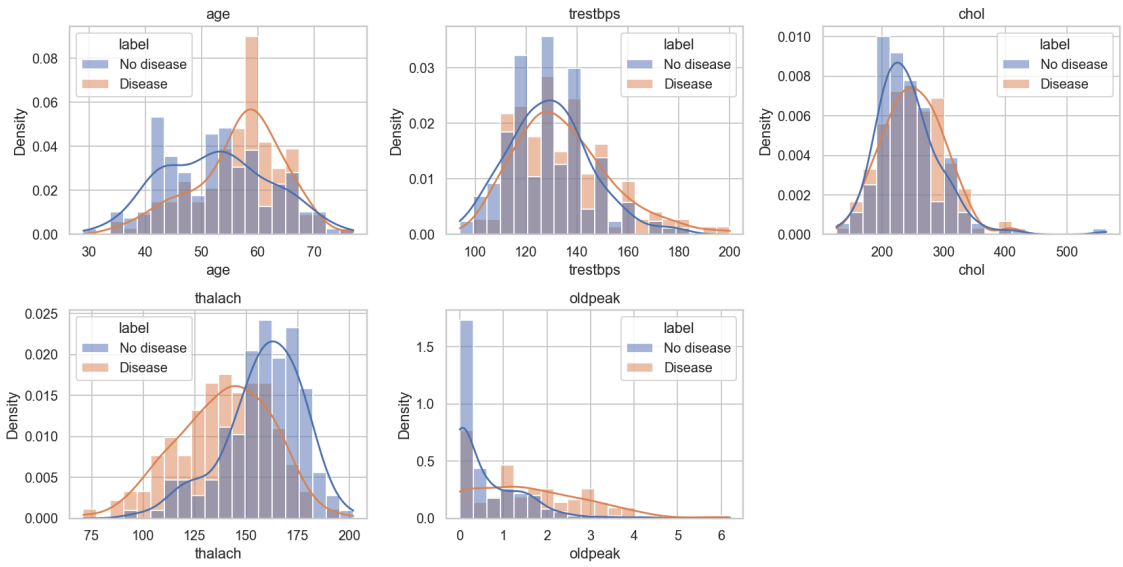
The raw file is downloaded from the UCI repository by src/data.py, so nobody has to supply data by hand. The cleaning step converts the '?' entries into missing values, casts all columns to numbers, removes duplicates and turns the original ‘target’ into a binary label. The original ‘target’ runs from 0 (healthy) to 4 (severe), but the task is simply disease or no disease, so values 1 to 4 become 1. A few values are missing in ‘ca’ and ‘thal’. They are not filled in during cleaning. They are imputed inside the model pipeline instead, so that the fill values are learned only from training data. Filling them beforehand would let information from the test set leak into training.

The EDA looks at the class balance, the distribution and outliers of the numeric features, the disease rate within each categorical feature, and the correlation between features. The figures are saved in reports/figures and are produced by the same code that the notebook runs.

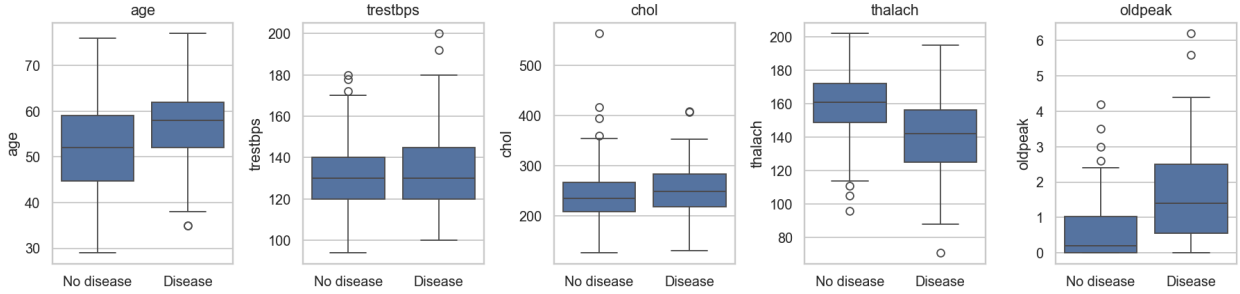
Reports:



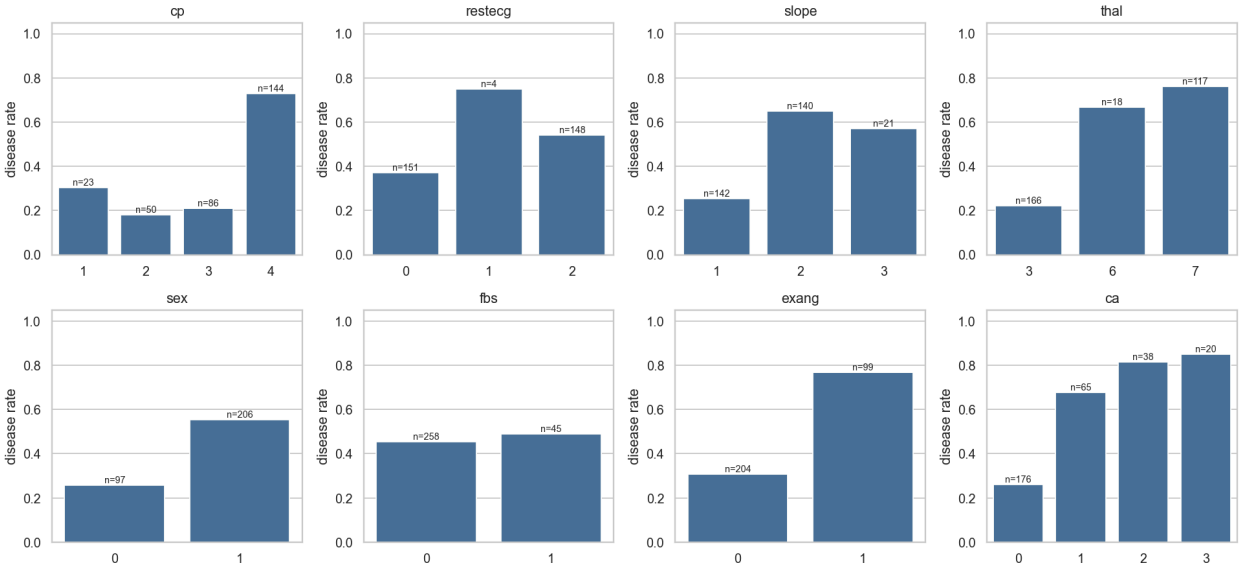
Distribution of healthy vs disease records



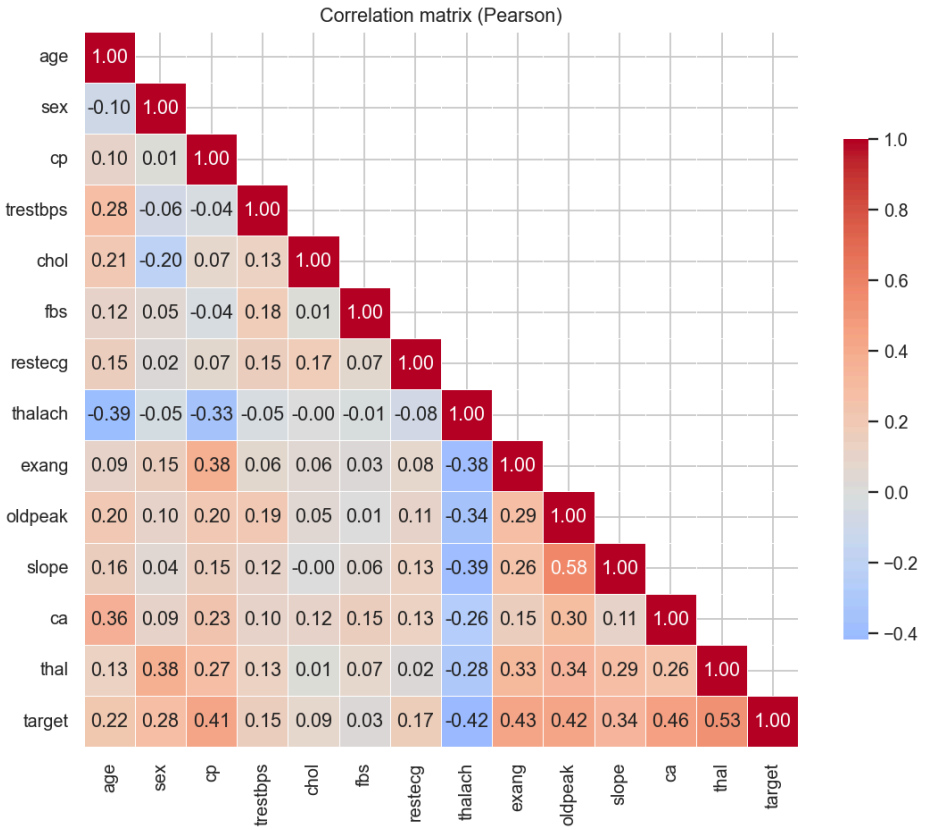
Distribution of the numeric features



Numeric features split by class (shows outliers and separation)



Disease rate within each categorical feature



Correlation between features

The classes are nearly balanced (164 healthy, 139 diseased). The strongest signals are ‘ca’ (disease rate rises from about 26% at 0 vessels to over 80% at 2 or more), ‘thal’ (about 76% disease for code 7 against 22% for normal), ‘cp’ = 4 (about 73%), ‘exang’, ‘oldpeak’ and a lower ‘thalach’. Age matters less, and ‘chol’, ‘trestbps’ and ‘fbs’ barely separate the classes. Outliers in ‘chol’ (one value above 550) and ‘trestbps’ (180 to 200) are plausible clinical readings, so I kept them. The 6 missing values (4 in ‘ca’, 2 in ‘thal’) are imputed inside the pipeline from training data only, to avoid test-set leakage.

4. Features and models

Two features are engineered on top of the 13 originals. ‘hr_pct_max’ is ‘thalach’ divided by (220 - ‘age’): it expresses the heart rate reached as a share of

the age-predicted maximum, which is more comparable between patients than the raw value. ‘oldpeak_x_exang’ multiplies ST depression by exercise-induced angina, so it is non-zero only when both signs occur together.

Preprocessing is part of the model pipeline. Numeric columns are median-imputed and scaled. The nominal columns ‘cp’, ‘restecg’, ‘slope’ and ‘thal’ are one-hot encoded because their numeric codes have no order. The binary and ordinal columns are imputed and scaled. A SelectKBest step then keeps only the most informative columns before the classifier. Because all of this sits inside one pipeline object, the API only has to load a single file, and the same transformations are applied in training and in serving.

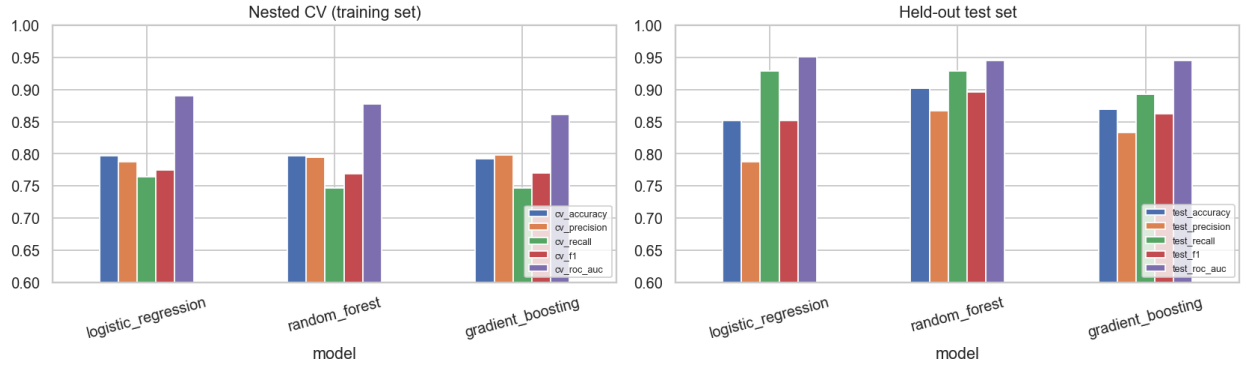
Three classifiers are compared: logistic regression, random forest and gradient boosting. The data is split 80/20 with stratification (seed 42), giving 61 test records. Each model is tuned and scored with nested cross-validation on the training part: an inner 3-fold search picks hyper-parameters, and an outer 5-fold loop scores the result. This keeps the reported scores honest, because the same data is never used both to tune a model and to judge it. The winner is chosen by nested-CV ROC-AUC.

Model	**CV acc**	**CV prec**	**CV recall**	**CV ROC-AUC**	**Test acc**	**Test recall**	**Test ROC-AUC**
Logistic regression	0.7971	0.7882	0.7644	0.8907	0.8525	0.9286	0.9513
Random forest	0.7972	0.7945	0.7466	0.8780	0.9016	0.9286	0.9448
Gradient boosting	0.7929	0.7982	0.7466	0.8609	0.8689	0.8929	0.9448

Logistic regression has the best cross-validated ROC-AUC (0.891) and the best test ROC-AUC (0.951), so it is selected, with C = 0.3, balanced class weights and k = 16 selected features. Random forest scores a higher test accuracy (0.902), but with only 61 test records a difference of a few patients is within noise, so the

more stable cross-validated AUC is the better guide. Recall is also high for the selected model (0.929 on the test set), which matters because missing a sick patient is worse than a false alarm.

I selected logistic regression because it had the best cross-validated and test ROC-AUC (0.891 and 0.951) and, with only 303 records, a simple linear model is less likely to overfit than random forest or gradient boosting. Its coefficients are also easy to interpret, which suits a clinical setting where a prediction has to be explainable.

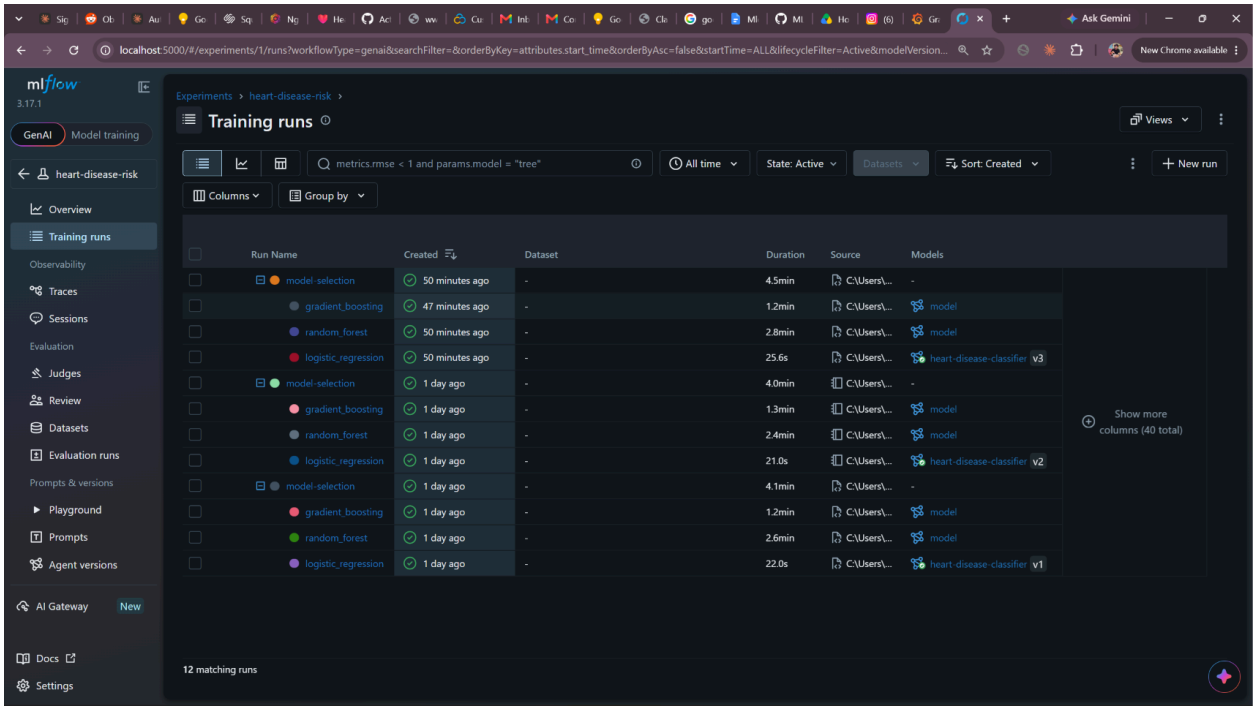


Comparison of model algorithms

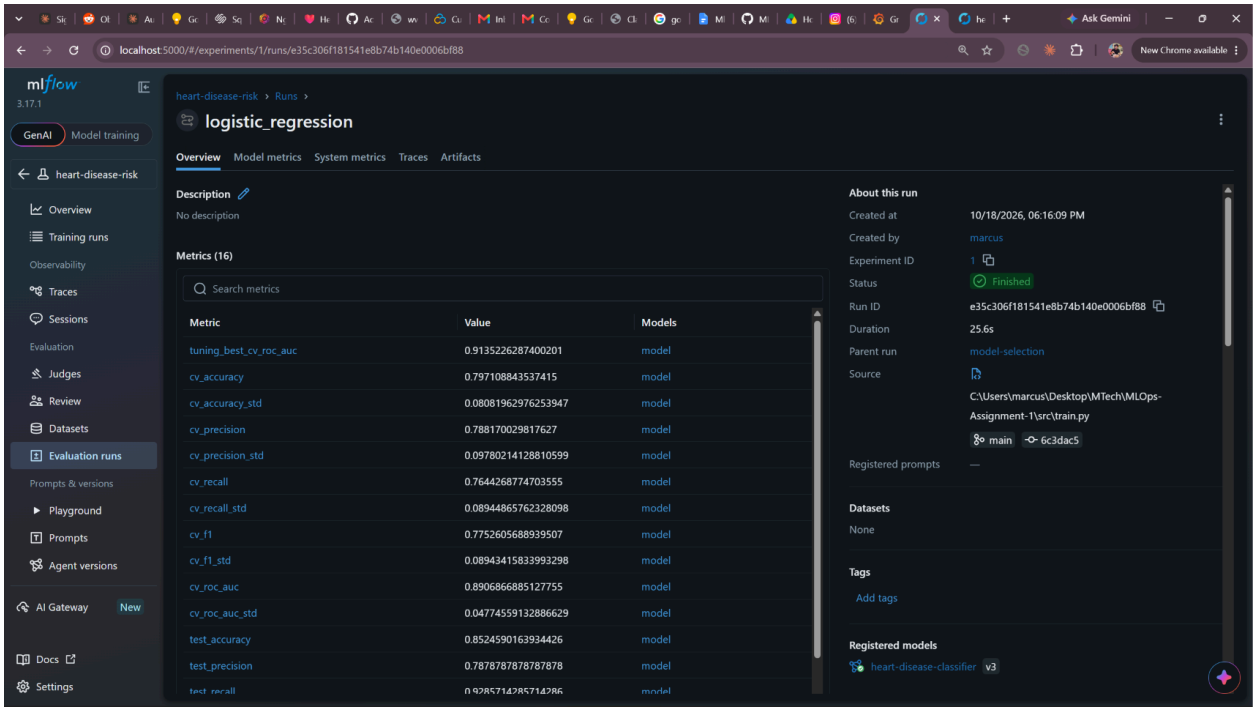
5. Experiment tracking

Every training run is logged to MLflow under the experiment heart-disease-risk. A SQLite file (mlflow.db) stores parameters and metrics, and a local folder (mlartifacts) stores files. Both are regenerated by python -m src.train, so they are not committed to git.

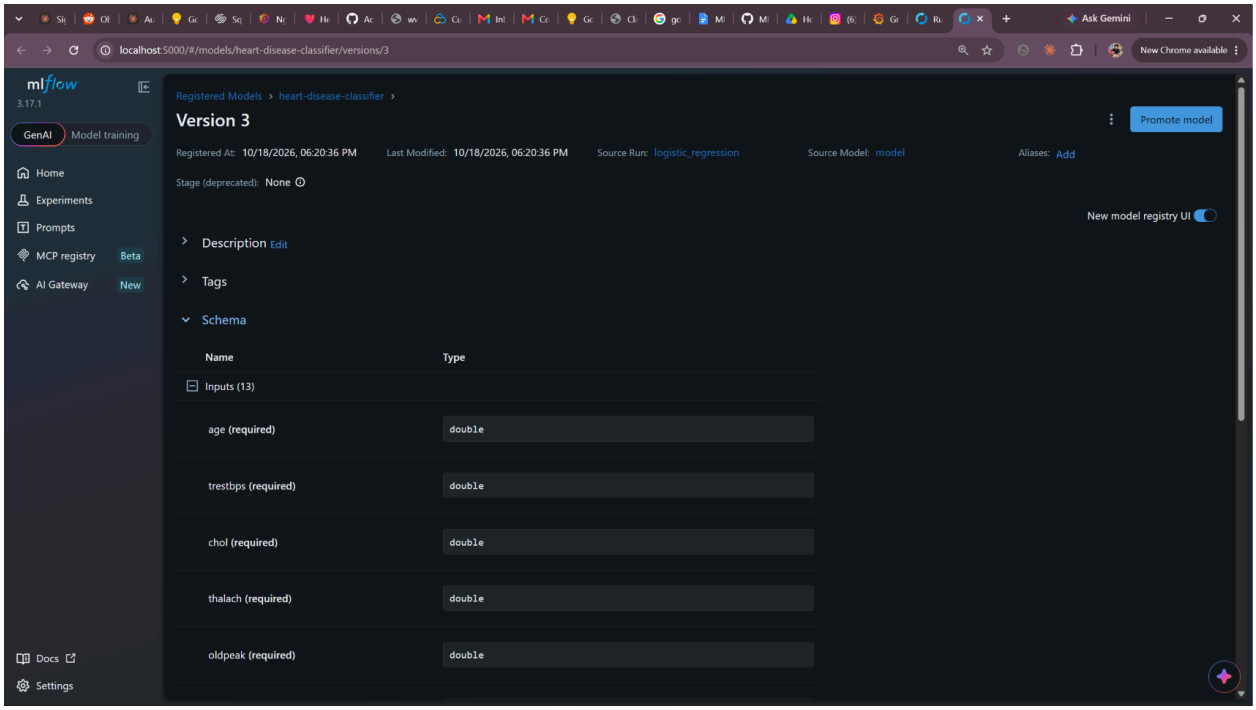
For each candidate model a run records the seed, test size and best hyper-parameters, the cross-validation and test metrics (accuracy, precision, recall and ROC-AUC), and plots such as the ROC curve and confusion matrix. This makes runs directly comparable in the MLflow UI. The selected model is registered as a heart-disease-classifier, so there is a named model that corresponds to the saved pipeline.



MLflow training runs list



MLflow individual run page



MLflow individual model page

6. Packaging and reproducibility

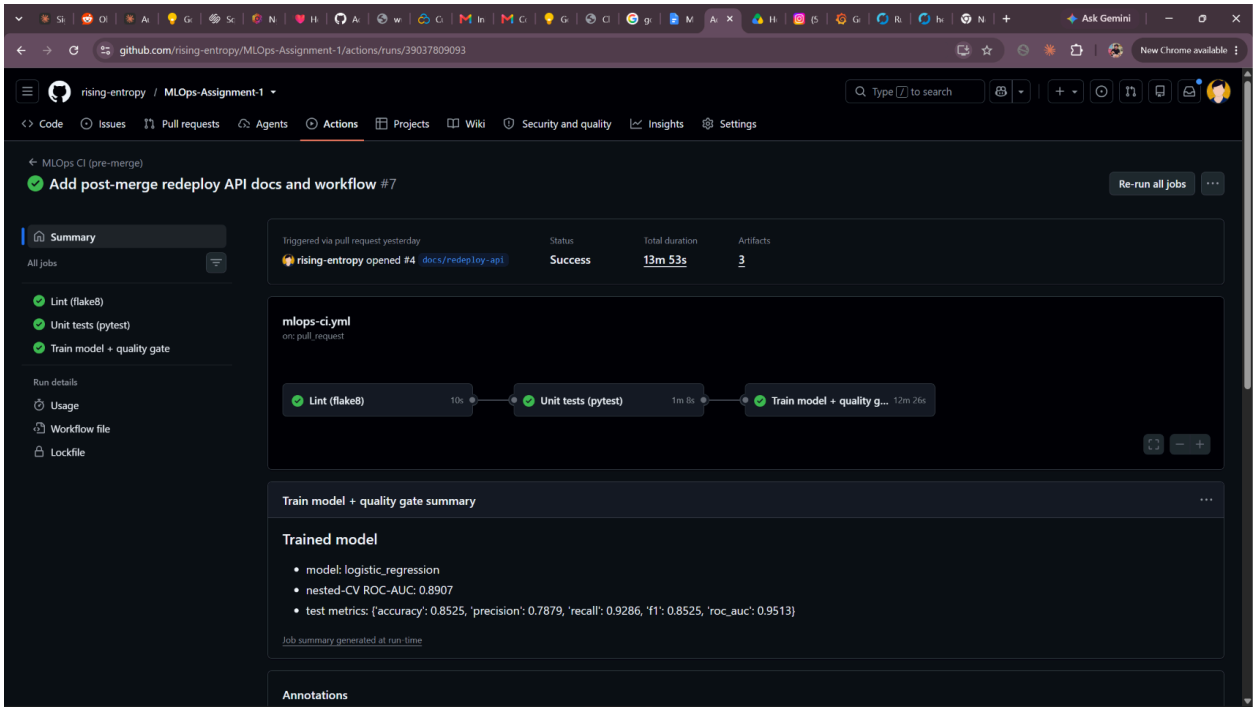
The aim is that the same inputs always produce the same model, on any machine.

- Seeds: the train/test split and both levels of cross-validation use fixed seeds (42 and 43), so results repeat exactly.
- Pinned versions: requirements.txt and api/requirements.txt fix exact versions. numpy, pandas, scikit-learn and joblib must match between training and the API image, because a pickled pipeline can fail to load, or predict differently, under other versions.
- One artifact: models/heart_model.joblib contains the whole pipeline, from feature engineering to the classifier. The committed copy means the server does not need to train anything.
- Shared code: the notebook, the tests, the API and CI all import the same src package, so no logic is duplicated.

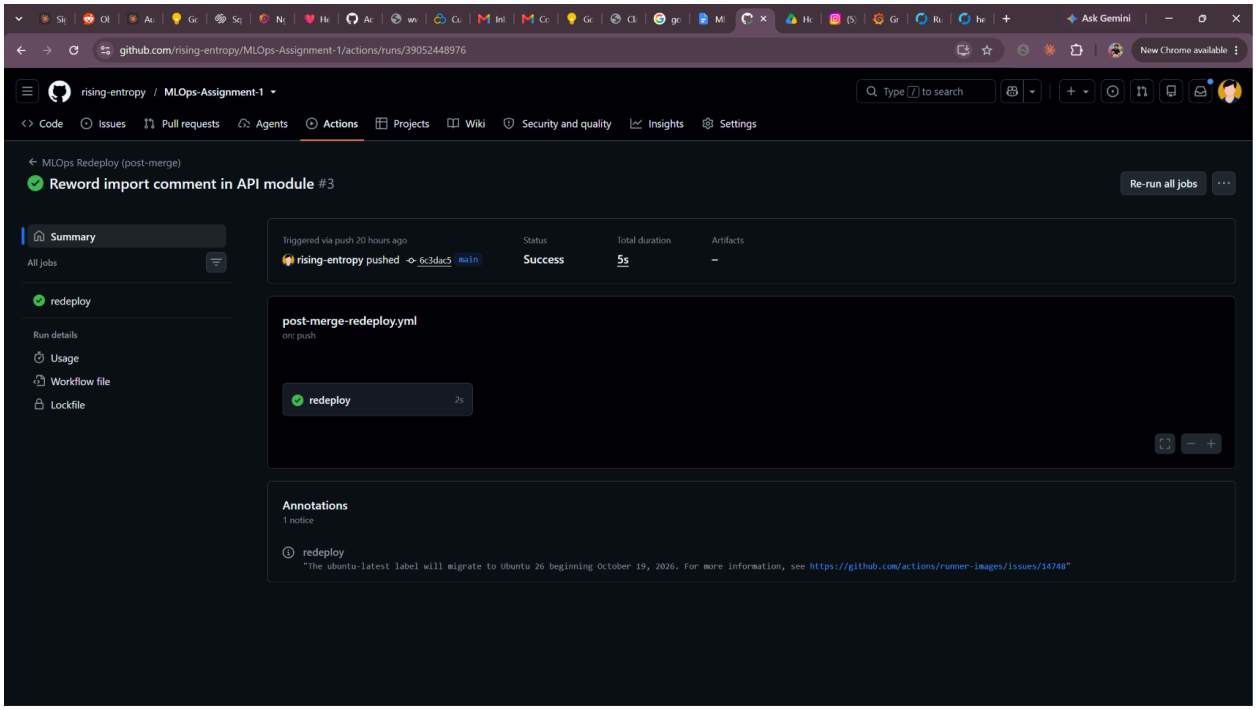
7. CI/CD

Continuous integration runs on every pull request into main. It has three stages in order: flake8 checks style in src and tests, pytest runs 31 tests on the data, feature and training code, and finally the model is retrained from scratch and passes a quality gate. The gate fails the build if the test ROC-AUC drops below 0.85 or recall below 0.80, so a change that makes the model worse cannot be merged unnoticed. Each run uploads its logs, the JUnit report, the trained model and the MLflow store as artifacts.

Continuous deployment runs when code lands on main. The workflow sends an authenticated request to a webhook on the server. The server then pulls the latest code and rebuilds and restarts the containers with docker compose up -d --build. The credentials are kept in GitHub secrets, and the redeploy service itself lives in a separate repository.



Pre-merge check run - [Link]



Post-merge pipeline - Link

8. Containerisation

The API image is built on Python 3.11-slim. It copies in the src package and the trained model, installs the pinned requirements and runs as a non-root user. It is served by gunicorn with one worker and four threads. A single worker is deliberate: the Prometheus counters live in process memory, and more workers would each report their own partial counts.

The API exposes four endpoints. GET /health reports liveness and is used by the Docker health check. GET / describes the service and the required features. POST /predict takes the 13 numeric features as JSON and returns the prediction, a label, a confidence and the probability of disease, and it answers HTTP 400 for missing or non-numeric input. GET /metrics exposes the Prometheus metrics. The UI image runs a Streamlit form that sends the same request to the API.

9. Deployment

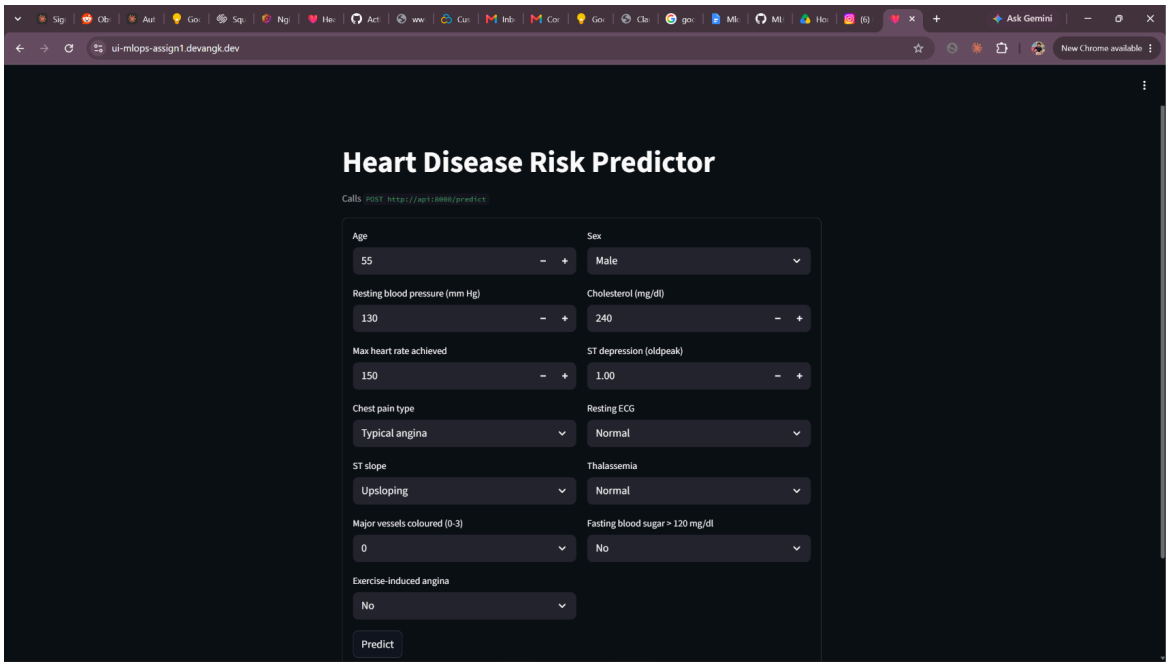
The application runs on a single VPS under Docker Compose. One docker-compose.yml defines the API, UI, Prometheus, Loki, Promtail and Grafana

on a shared network, with named volumes so monitoring data survives restarts. Merging to main triggers the redeploy described in section 7, so the live system always follows the repository. The services are available at:

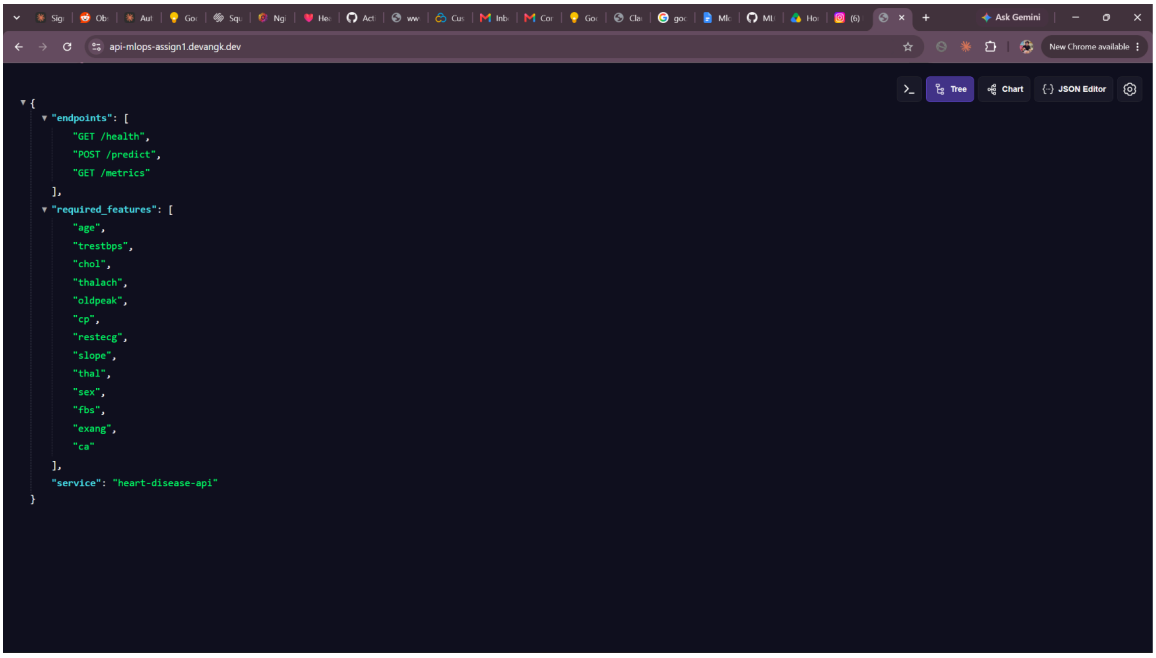
- UI: ui-mlops-assign1.devangk.dev
- API: api-mlops-assign1.devangk.dev
- Grafana dashboard: dash-mlops-assign1.devangk.dev
- Prometheus: metrics-mlops-assign1.devangk.dev

The assignment asks for a cloud or Kubernetes deployment with a manifest or Helm chart and an Ingress or load balancer. This project uses Docker Compose on a VPS and does not yet include Kubernetes manifests. Separately, Grafana is configured with a default admin login and anonymous viewing in docker-compose.yml, and Prometheus is publicly queryable.

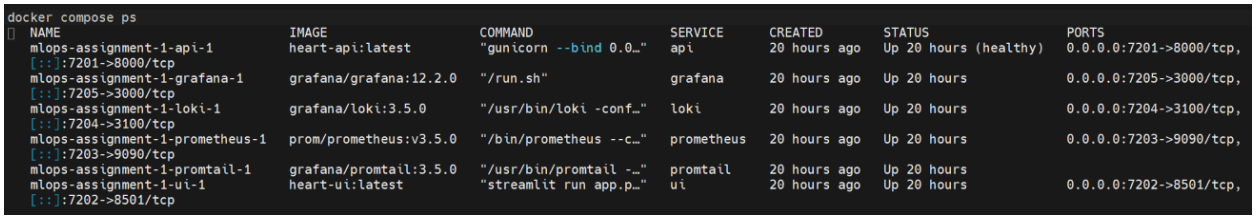
I went with this approach over k8s since the deployment was for a demo scale and across single server, for our use-case to build a CI/CD pipeline for this work, where we didn't have expectation of 0 downtime, or a rolling update, this approach is simple, yet scalable with a downtime of less than milli-seconds per update. The helm-chart equivalent here is the 'docker-compose.yml' file where we define all the images that would be deployed on the server.



Web UI interface



API endpoint



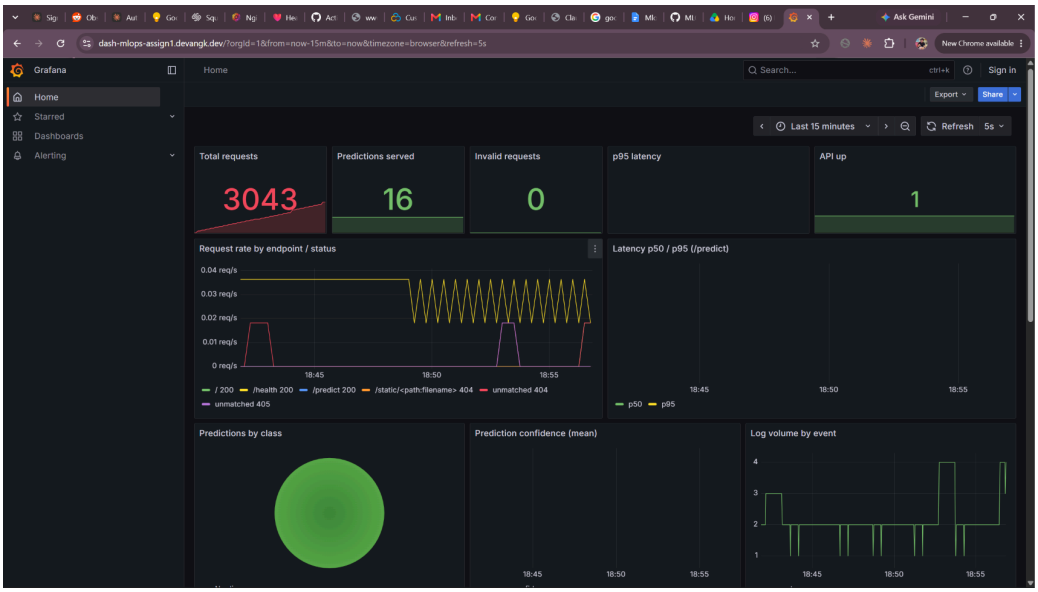
Server compose output

10. Monitoring and logging

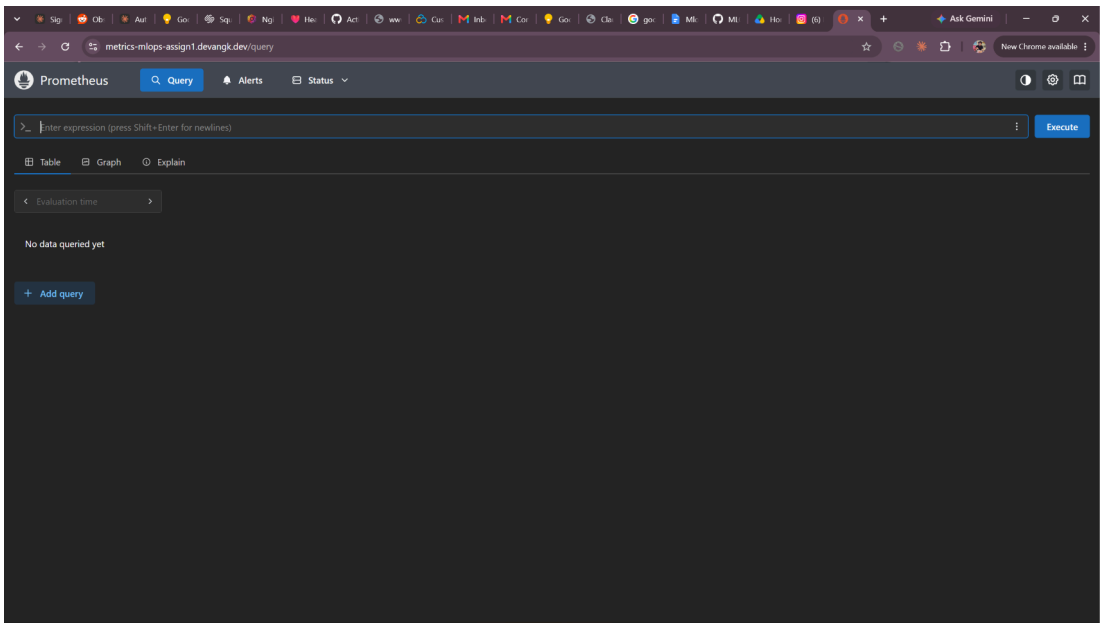
The API writes one JSON object per line to standard output for each request, prediction, invalid request and model load. Promtail reads the container logs through the Docker socket and ships them to Loki, which makes them searchable by event type in Grafana.

Prometheus scrapes the API's /metrics endpoint every 5 seconds. The metrics are request counts by method, endpoint and status, request latency, predictions by label, prediction confidence and the number of invalid requests. A Grafana dashboard, provisioned from files so it exists on a fresh deployment, shows these metrics next to the logs.

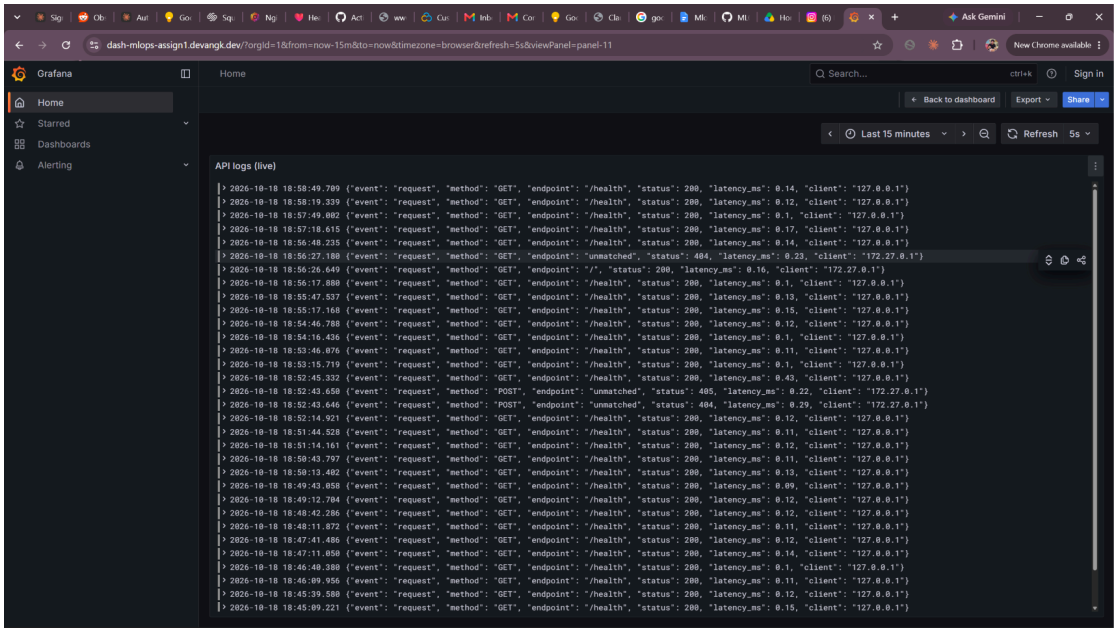
One privacy concern: the API logs the prediction inputs. This is acceptable for a public benchmark dataset, but the inputs would need to be dropped or hashed before real patient data was used.



Grafana dashboard



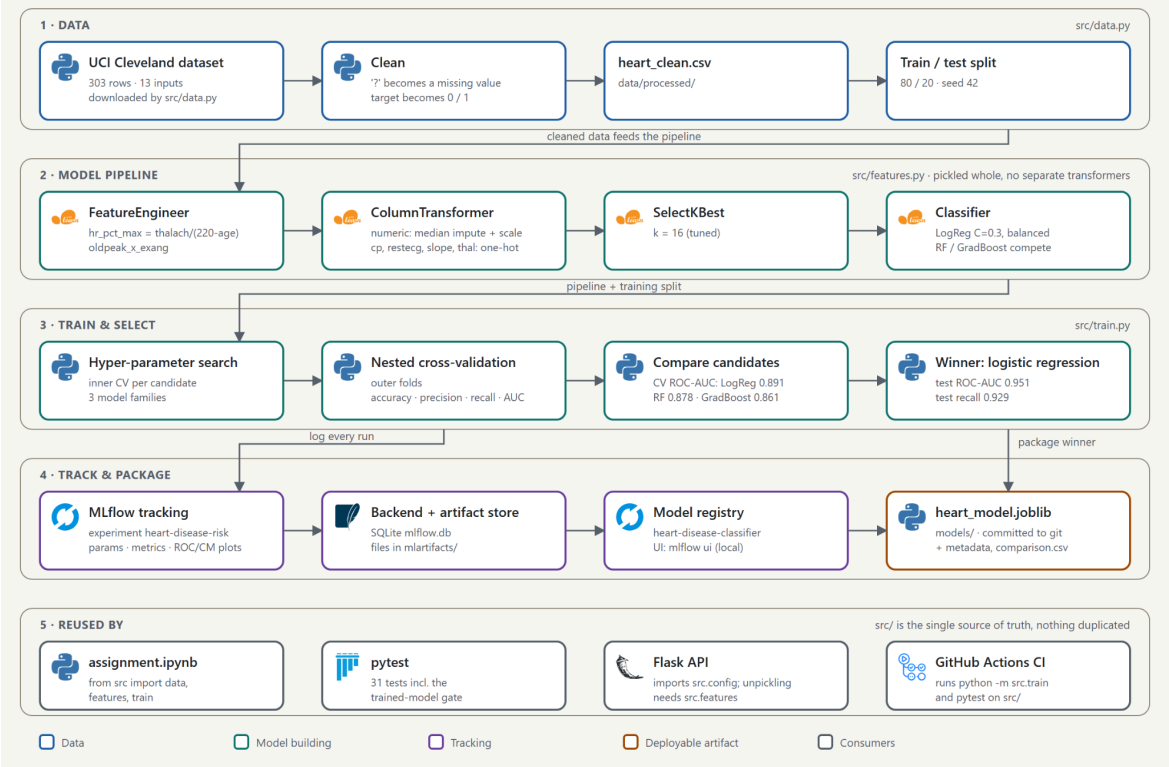
Prometheus metrics UI



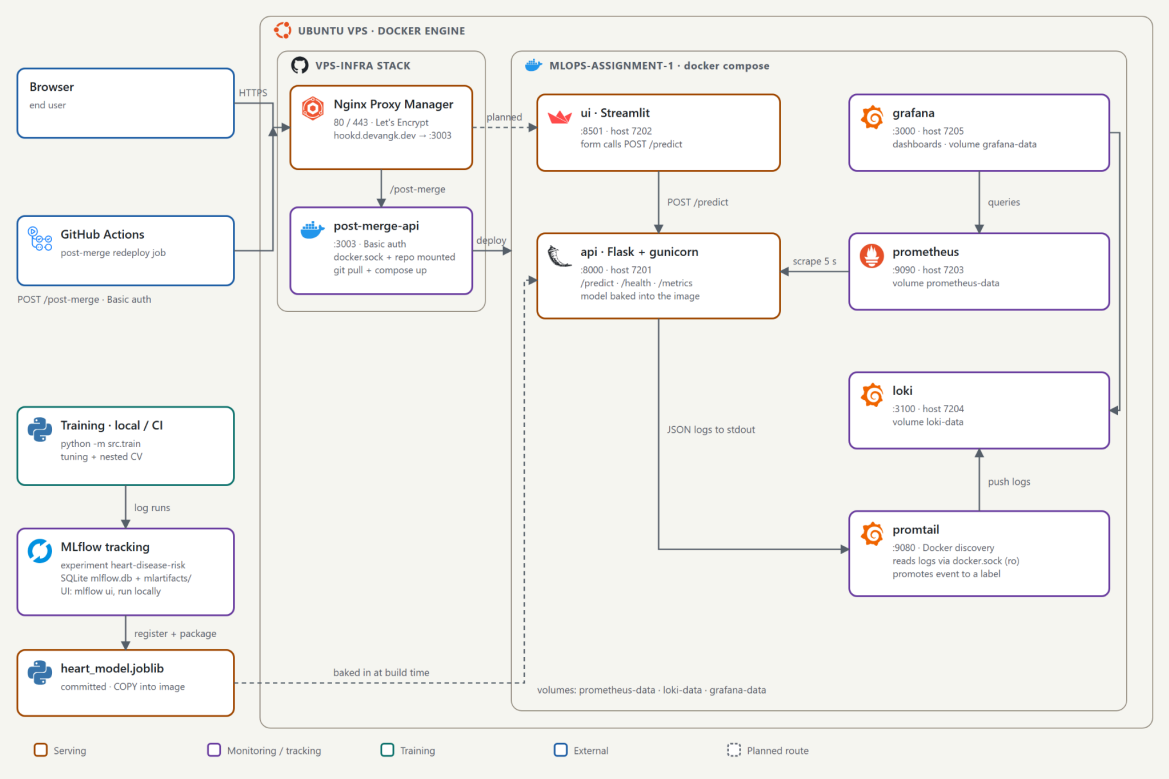
Live logs in Grafana

11. Architecture

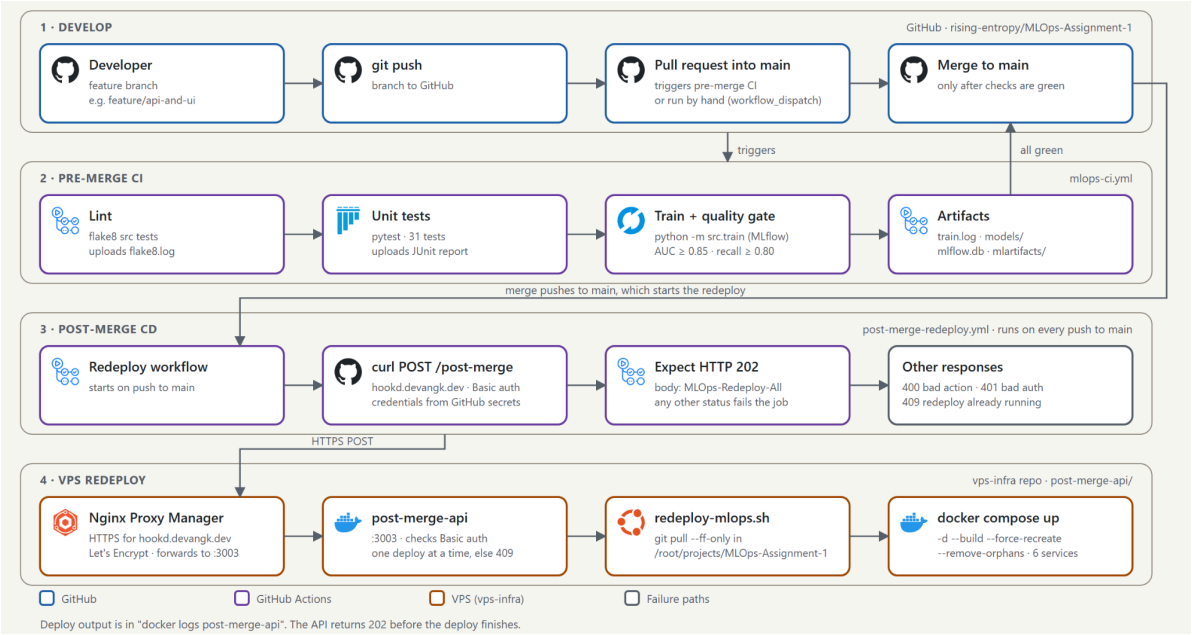
The system has an offline and an online part. Offline, the src package cleans the data, builds features and trains the models; MLflow records each run, and the winning pipeline is saved as a single file. Online, the Streamlit UI sends requests to the Flask API, which loads that file at start-up. Prometheus scrapes the API, Promtail forwards its logs to Loki, and Grafana reads from both. GitHub Actions checks every pull request and redeploys the server when code is merged.



Model Architecture Diagram - [Link]



System Architecture Diagram - [Link]



CI/CD Pipeline Diagram - Link

12. Conclusion and references

The project covers the full lifecycle: a reproducible training pipeline, tracked experiments, automated checks with a quality gate, containers, an automatically redeployed server and live monitoring.

This project made me realize that ML system architecture goes way before just model building, intricate things like experiment tracking which for small projects don't matter, but in production systems, is very helpful and necessary is indeed a good learning experience. I also realized that the ops is just as difficult and tricky as the ML model building part if not more - managing constraints and optimizing.

References

- Dua, D. and Graff, C. UCI Machine Learning Repository: Heart Disease. https://archive.ics.uci.edu/ml/datasets/heart+disease
- Pedregosa et al. Scikit-learn: Machine Learning in Python. JMLR 12, 2011. https://jmlr.org/papers/v12/pedregosa11a.html
- MLflow documentation. https://mlflow.org/docs/latest/
- Prometheus documentation. https://prometheus.io/docs/
- Grafana Loki documentation. https://grafana.com/docs/loki/latest/
- Grafana documentation. https://grafana.com/docs/grafana/latest/